Benchmark des modèles locaux (Ollama, CPU) — étape 3

Piste : « 3 questions qui débloquent les briefs ». Mis à jour le 11/11/2026 : les 8 modèles prévus sont tous mesurés. Chiffres bruts : `benchmark/runs/<pc>.json` *; tableaux régénérés par* `python benchmark/run_benchmark.py --rapport` *→ tableaux.md.*

> **Prudence :** ces résultats reposent sur **15 cas seulement**, et le **garde-fou a été choisi après avoir vu les résultats**. Ils indiquent une tendance, pas une preuve. La **validation est prévue jeudi 12/11 sur les 30 cas de référence** définis avec Product, sans retoucher les prompts ni le garde-fou.

1. Ce qu'on mesure, et pourquoi

Tâche	Usage dans le produit	Cas de test	Mesures
Recherche : besoin du commercial en langage courant → bonne question de la banque (24 questions)	Le junior tape « je ne sais pas quand ils lancent » et retrouve la question qui marche	15 besoins, avec la rubrique et la ou les questions attendues	Bonne question au **rang 1**, dans le **top 3** (les 3 questions montrées au junior), dans le **top 5**, MRR, **temps** par requête
Extraction (LLM) : note du rendez-vous précédent → rubriques du brief déjà connues, en JSON	Savoir quelles rubriques sont encore vides, pour choisir les 3 questions (entrée `rubriques_connues` de `POST /questions`)	15 notes, avec les rubriques attendues	**JSON valide**, **F1** (brut et avec garde-fou), notes avec **rubrique inventée**, **temps** par note

Méthode :

- Les 15 cas (briefs_test.csv) ont été écrits **avant** le premier prompt et n'ont pas été modifiés ensuite.
- Les besoins sont volontairement formulés avec d'autres mots que les questions de la banque, pour tester la compréhension du sens.
- Réglages : Ollama sur `127.0.0.1`, `temperature 0`, graine 42, au plus 400 tokens par réponse, vecteurs de la banque en cache.
- Préfixes : nomic-embed-text et mxbai-embed-large utilisent le préfixe de requête recommandé par leur documentation (`search_query:` pour nomic, qui ajoute aussi `search_document:` devant les questions de la banque ; « Represent this sentence for searching relevant passages: » pour mxbai). Les autres modèles n'utilisent aucun préfixe.
- Le TF-IDF sert de référence sans IA.
- Les 6 premiers modèles ont été mesurés sur le même PC, dans le même passage (11/11/2026). mxbai-embed-large et llama3.2:3b viennent d'un passage complémentaire du 11/11, sur la même machine et avec les mêmes réglages (mêmes 15 cas, même garde-fou). Les temps de réponse peuvent légèrement varier d'une session à l'autre.
- Machine (PC principal) : processeur AMD Ryzen 5 PRO 4650U (6 cœurs / 12 threads), 15,3 Go de RAM (1,9 à 2,2 Go libres au lancement des passages), **pas de GPU dédié** (graphique intégré AMD Radeon, 512 Mo) → calculs sur CPU uniquement ; Windows 11 Pro, Ollama 0.40.16, Python 3.11.9. RAM, cœurs et versions :

`runs/pc_principal.json` ; nom du processeur et carte graphique : relevés sous Windows.

2. Contrainte matérielle et solution

	PC principal
Processeur	AMD Ryzen 5 PRO 4650U (6 cœurs / 12 threads), carte graphique intégrée → **CPU uniquement**
RAM	15,3 Go au total, mais **seulement 0,6 à 2,4 Go libres** pendant les tests (navigateur, VS Code, API ouverts)
Disque C:	presque plein : 0,3 Go libres le 10/11, 1,6 Go le 11/11 (sur 237 Go)
Disque D:	~119 Go libres
Système	Windows 11 Pro, Ollama 0.40.16, Python 3.11.9

Le problème : Ollama range ses modèles sur C: par défaut. Le 10/11, seuls `nomic-embed-text` et `qwen2.5:1.5b` y tenaient. `all-minilm`, `bge-m3`, `qwen2.5:0.5b` et `qwen2.5:3b` n'avaient pas pu être téléchargés.

La solution : une deuxième instance d'Ollama, avec ses modèles sur D:

Étape 1. Une instance d'Ollama, c'est un serveur. On peut en lancer un deuxième, à côté de l'habituel, en lui donnant un autre port et un autre dossier pour les modèles, grâce à deux variables d'environnement :

```
$env:OLLAMA_MODELS = 'D:\ollama\models'   # où ranger les modèles
$env:OLLAMA_HOST   = '127.0.0.1:11435'    # autre port que l'habituel (11434)
ollama serve
```

Étape 2. On télécharge les modèles dans cette instance : `ollama pull …`, avec `OLLAMA_HOST=127.0.0.1:11435`. Ça représente environ 4,8 Go, tous écrits sur D:.

Étape 3. On lance le benchmark en lui indiquant cette instance : `OLLAMA_HOST=http://127.0.0.1:11435`.

Avantages :

- l'Ollama habituel (port 11434, modèles sur C:) n'est pas touché ;
- aucun dossier n'est déplacé et aucun réglage système n'est modifié ;
- on revient à l'état initial simplement en fermant la deuxième instance.

Limite qui reste : la RAM.

- Avec environ 2 Go libres, Windows déplace une partie de la mémoire sur le disque dès qu'un modèle dépasse cette taille. C'est le cas de `qwen2.5:3b` (1,95 Go en mémoire) : il met jusqu'à 62 s pour une note, alors que la moyenne est de 24,5 s.
- Les **temps** mesurés sont donc des ordres de grandeur, et ils seraient meilleurs sur un PC moins chargé.
- Les **scores de qualité**, eux, ne dépendent pas de la RAM (temperature 0).

3. Résultats — recherche de la bonne question (15 cas)

Modèle	Taille disque	Mémoire chargé	Bonne question au rang 1	Bonne question dans le top 3	Bonne question dans le top 5	MRR	Temps / requête	Testé sur
TF-IDF (référence, sans IA)	0	~0	33 % (5/15)	47 % (7/15)	53 % (8/15)	0,44	**0,1 ms**	PC principal
nomic-embed-text	274 Mo	0,36 Go	47 % (7/15)	60 % (9/15)	73 % (11/15)	0,56	100–130 ms	PC principal
all-minilm	45 Mo	**0,05 Go**	67 % (10/15)	80 % (12/15)	87 % (13/15)	0,76	**26–40 ms**	PC principal
bge-m3	1,2 Go	1,16 Go	**93 % (14/15)**	**100 % (15/15)**	**100 % (15/15)**	**0,96**	150–300 ms	PC principal
mxbai-embed-large	669 Mo	0,69 Go	40 % (6/15)	60 % (9/15)	80 % (12/15)	0,56	~220 ms	PC principal

Indexation de la banque, faite une seule fois puis mise en cache : 0,8 s (all-minilm), 2,4 s (nomic), 10,4 s (bge-m3), 11,8 s (mxbai-embed-large).

Lecture :

- Sur 15 besoins, bge-m3 met la bonne question dans les 3 proposées **15 fois**, contre 12 pour all-minilm, 9 pour nomic-embed-text et mxbai-embed-large, et 7 pour le TF-IDF.
- mxbai-embed-large, plus gros que nomic, ne fait pas mieux au top 3 (9/15) et il n'est pas plus rapide que bge-m3 (~220 ms par requête, dans la fourchette de bge-m3 : 150–300 ms).
- bge-m3 est **multilingue**, ce qui explique sans doute son avance sur des phrases en français.

4. Résultats — extraction JSON des rubriques connues (15 notes)

Le garde-fou :

- C'est une règle simple, sans IA, ajoutée **après** le premier passage : une rubrique ne compte que si sa valeur est non vide et si ses mots se retrouvent dans la note.
- Comme elle a été décidée en voyant ces résultats, elle **devra être confirmée sur les 30 cas de validation** de l'étape 4.

Modèle	Taille disque	Mémoire chargé	JSON valide	F1 brut	F1 avec garde-fou	Notes exactes (garde-fou)	Notes avec rubrique inventée (brut → garde-fou)	Temps / note (moy. / max)	Testé sur
qwen2.5:0.5b	397 Mo	0,43 Go	73 %	0,34	0,41	7 % (1/15)	73 % → 47 %	**6,7 s** / 12,5 s	PC principal
qwen2.5:1.5b	986 Mo	1,04 Go	**100 %**	0,59	0,72	7 % (1/15)	100 % → 87 %	8,3 s / 11,4 s	PC principal
qwen2.5:3b	1,9 Go	1,95 Go	93 %	0,52	**0,81**	**47 % (7/15)**	93 % → **20 %**	24,5 s / 62,1 s	PC principal
llama3.2:3b	2,0 Go	2,33 Go	**100 %**	0,62	**0,82**	20 % (3/15)	87 % → 40 %	17,0 s / 27,6 s	PC principal

Lecture :

- **Sans garde-fou, les modèles cochent presque toutes les rubriques.** Leur F1 brut reste faible (0,34 à 0,62).
- **llama3.2:3b fait jeu égal avec qwen2.5:3b sur le F1 avec garde-fou** (0,82 contre 0,81) et il est plus rapide (17,0 s par note contre 24,5 s). Mais il a moins de notes exactes (3/15 contre 7/15) et invente encore une rubrique dans 40 % des notes après garde-fou (contre 20 %). Il prend aussi plus de mémoire (2,33 Go contre 1,95 Go).
- **Le garde-fou profite surtout au 3b** : ses valeurs reprennent le texte de la note, donc la règle filtre mieux ses inventions. Il passe à 47 % de notes exactes, contre 7 % pour les deux autres.
- **Le 0.5b est le moins fiable :**
  - plus d'un JSON sur quatre est invalide ;
  - il invente des codes de rubrique qui n'existent pas (« valideurs », « valores ») ;
  - lors d'un premier passage, il a bouclé sans fin (plus de 17 000 tokens pour une note). D'où le plafond de 400 tokens par réponse, ajouté pour tous les modèles.

5. Trois exemples concrets

Exemple 1 : les embeddings comprennent le sens, le TF-IDF compare les mots (cas T14).

- Besoin : « Quel public vise-t-on : les jeunes, les familles ou le haut de gamme ? »
- TF-IDF : il propose en n°1 Q02 (« création, rénovation ou extension de gamme ? »), une question hors sujet.
- nomic-embed-text, all-minilm et bge-m3 proposent en n°1 Q05 (« plutôt un public jeune, familial ou premium ? »), la bonne réponse, alors que « haut de gamme » et « premium » ne partagent aucun mot. mxbai-embed-large la place au rang 2, derrière Q12.

Exemple 2 : un piège de vocabulaire que seul bge-m3 évite (cas T09).

- Besoin : « Quand est-ce que ça doit arriver dans les magasins ? »
- La bonne réponse est Q19 (« à quelle date le produit doit-il être en rayon ? »).
- nomic-embed-text, all-minilm et mxbai-embed-large proposent Q17 (« à quel prix le produit fini sera-t-il vendu **en magasin** ? »). Le mot « magasin » les attire : Q19 arrive seulement au rang 15 pour nomic et mxbai, et au rang 10 pour all-minilm.
- TF-IDF : Q19 au rang 3, derrière la question générique Q21.
- **bge-m3 : Q19 au rang 1.** Il a compris que la question porte sur une **date**, pas sur un prix.

Exemple 3 : les LLM inventent, le garde-fou limite les dégâts (cas T03).

- Note : « Lancement prévu en septembre 2027 avec trois tours de validation. Ils n'ont pas encore d'idée précise sur le produit. » La seule rubrique attendue est `delai`.
- **Brut :** les trois modèles qwen2.5 cochent 7 ou 8 rubriques. Le 1.5b invente `"cible": "femme"` et prend le nom du client (« Ondevia ») pour une référence. Le 0.5b invente une rubrique `valideurs`.
- **Avec le garde-fou :**
  - il reste `delai` + `references` pour le 1.5b ;
  - `delai` + `type_produit` pour le 3b ;
  - toujours 8 rubriques pour le 0.5b ;
  - `delai` + `marche` pour llama3.2:3b, qui n'en cochait que ces deux-là au départ : il a pris « septembre 2027 » pour un marché.
- **Aucun n'est exact sur ce cas.** La règle ne détecte pas une erreur de sens quand le mot est bien écrit dans la note.

6. Recommandation finale pour la V2

Le contexte d'usage : le commercial prépare son rendez-vous sur un **PC portable de bureau**, sans carte graphique, avec en général 8 à 16 Go de RAM, dont une bonne partie déjà prise par Outlook, Teams, le navigateur et le CRM. Il faut donc rester léger en mémoire et rapide pour tout ce qui est **interactif**. Ce qui peut être calculé à l'avance peut, en revanche, prendre plus de temps.

Brique	Choix V2	Qualité	Temps	Mémoire	Pourquoi
Recherche d'une question (interactive)	**bge-m3**	100 % au top 3	0,15–0,3 s	1,2 Go	Le meilleur, avec une marge nette (+20 points sur all-minilm), et assez rapide pour taper une requête
Repli n°1 (PC avec moins de 4 Go libres)	**all-minilm**	80 % au top 3	~30 ms	0,05 Go	Presque gratuit en mémoire, encore très correct
Repli n°2 (Ollama absent)	**TF-IDF**	47 % au top 3	0,1 ms	~0	Ne dépend de rien : l'outil ne tombe jamais en panne
Rubriques déjà connues (préparation)	**qwen2.5:3b + garde-fou**, en **pré-remplissage** à confirmer	F1 0,81, 47 % exact	~25 s par note, en arrière-plan	~2 Go, libérés après usage	Le moins inventif, mais pas assez fiable pour décider seul
Pas assez de RAM pour le 3b	**pas de LLM** : le junior coche lui-même les rubriques connues	—	—	0	Le 1.5b invente une rubrique dans 87 % des notes : mieux vaut ne rien pré-remplir que pré-remplir faux

Alternative : llama3.2:3b, plus rapide (17,0 s par note contre 24,5 s, maximum 27,6 s contre 62,1 s) avec un F1 équivalent (0,82 contre 0,81), mais il invente deux fois plus de rubriques après garde-fou (40 % contre 20 %). Sur 15 cas, l'écart de F1 n'est pas significatif : les deux modèles seront comparés sur les 30 cas de validation.

En pratique dans la V2 (`src/moteur_v2.py`) :

1. **Recherche :** bge-m3. Si le modèle est absent ou si la RAM libre est sous 4 Go, la V2 passe sur all-minilm, puis sur TF-IDF si Ollama ne répond pas. La sortie garde le même format que la V1, avec un champ qui indique la méthode utilisée.
2. **Extraction :**
   - le 3b tourne **une fois par rendez-vous, à la préparation**, pas pendant la saisie ;
   - Ollama décharge le modèle de la mémoire après quelques minutes ;
   - le junior voit les rubriques proposées et les **valide en un clic**, ce qui va dans le sens du produit.
3. **Critère honnête :** le critère fixé à l'avance pour une extraction automatique (F1 d'au moins 0,85 et moins de 20 s par note) **n'est pas atteint**. C'est pour ça qu'on choisit le pré-remplissage plutôt que l'automatisme.
4. **À confirmer sur les 30 cas de validation (étape 4) :** l'avance de bge-m3 (15 cas, c'est peu), l'effet du garde-fou, et une recherche hybride TF-IDF + bge-m3.

7. Limites (à dire au jury)

- **15 cas seulement :** un cas représente 6,7 points de pourcentage. Le 100 % de bge-m3 est un très bon signe, pas une garantie : il faut le confirmer sur les 30 cas de validation.
- **Même auteur pour la banque et les cas de test** (données fictives) : risque de biais de formulation.
- **Le garde-fou et le plafond de 400 tokens ont été ajoutés après avoir vu des résultats.** Ce sont des règles générales, pas un réglage du prompt, mais il faut les confirmer sur les cas de validation.
- **RAM limitée (environ 2 Go libres)** : les temps, surtout ceux du 3b, sont gonflés par l'usage du disque à la place de la mémoire. Ce sont des ordres de grandeur mesurés sur un seul PC, en un seul passage.
- **Mémoire « chargé »** : mesurée avec `ollama ps` et un contexte de 2 048 tokens. Un contexte plus long demande plus de mémoire.
- **Deux passages :** mxbai-embed-large et llama3.2:3b ont été mesurés dans un passage complémentaire du 11/11 (même machine, mêmes réglages). Les temps peuvent légèrement varier d'une session à l'autre.
- La recherche ne voit que le texte des questions : elle ne tient pas encore compte du score de réussite des questions (c'est le rôle du moteur V1).

8. Reproduire

```
# Si le disque C: est plein : 2e instance d'Ollama avec les modèles sur D: (PowerShell)
$env:OLLAMA_MODELS='D:\ollama\models'; $env:OLLAMA_HOST='127.0.0.1:11435'; ollama serve
# Dans un autre terminal :
$env:OLLAMA_HOST='127.0.0.1:11435'
ollama pull nomic-embed-text; ollama pull all-minilm; ollama pull bge-m3
ollama pull qwen2.5:0.5b; ollama pull qwen2.5:1.5b; ollama pull qwen2.5:3b
ollama pull mxbai-embed-large; ollama pull llama3.2:3b
cd data-brique
$env:OLLAMA_HOST='http://127.0.0.1:11435'
python benchmark/run_benchmark.py --pc pc_principal --embed nomic-embed-text all-minilm bge-m3 mxbai-embed-large --llm qwen2.5:0.5b qwen2.5:1.5b
qwen2.5:3b llama3.2:3b
python benchmark/exporter_pdf.py      # -> benchmark_llm.pdf
python benchmark/run_benchmark.py --rapport   # régénère tableaux.md
```